

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Тульский государственный университет»

Институт прикладной математики и компьютерных наук
Кафедра «Вычислительная техника»

Утверждено на заседании кафедры
«Вычислительная техника»
№7 от 26.01.2022

Заведующий кафедрой

_____ А.Н. Ивутин

**Методические указания по выполнению курсовой работы по дисциплине
«Системное программное обеспечение»**

**основной профессиональной образовательной программы
высшего образования – программы бакалавриата**

по направлению подготовки
09.03.01 «Информатика и вычислительная техника»

с профилем
«Электронно-вычислительные машины, комплексы, системы и сети»

Формы обучения: очная, заочная

Идентификационный номер образовательной программы: 090301-02-22

Тула 2022 год

ЛИСТ СОГЛАСОВАНИЯ
рабочей программы дисциплины (модуля)

Разработчик

Савин Н.И., доцент, к.т.н.

1 ЦЕЛЬ И ЗАДАЧИ ВЫПОЛНЕНИЯ КУРСОВОЙ РАБОТЫ

Курсовая работа предназначена для закрепления теоретических знаний в области проектирования программного обеспечения ЭВМ и получения практических навыков составления и отладки грамматик алгоритмических языков и реализации основных компонент трансляторов.

2 ТРЕБОВАНИЯ К КУРСОВОЙ РАБОТЕ

2.1 Тематика курсовой работы

Курсовая работа выполняется в полном соответствии с вариантом задания. Задание выдается в начале семестра, срок выполнения работы не превышает одного семестра.

В работе должны быть разработаны:

- грамматика языка программирования;
- лексический анализатор;
- синтаксический анализатор;
- семантический анализатор;
- программа генерации объектного кода.

Объектный код одинаковый для всех вариантов - Макроассемблер IBM PC.

Курсовая работа оформляется в соответствии с действующими стандартами на оформление программной документации и требованиями кафедры ЭВМ.

В пояснительной записке приводятся следующие разделы:

- техническое задание на проектирование;
- обзор литературных источников;
- постановка задачи на проектирование;
- разработка грамматики языка программирования;
- разработка лексического анализатора;
- разработка синтаксического анализатора;
- разработка семантического анализатора;
- разработка программ генерации объектного кода;
- исходный текст программы транслятора;
- тестовые примеры с текстами промежуточного кода, лексической свертки, диагностикой ошибок.

Курсовая работа представляется на защиту в законченном виде, полностью работоспособной и правильно оформленной.

2.2 Перечень вариантов заданий

Номер варианта рассчитывается по формуле

$$nv = \sum_{i=0}^{nd} a_i \bmod (cv) + 1,$$

где

nv- номер варианта ($0 < nv \leq cv$);

cv - количество вариантов ;

nd - количество цифр в номере договора;

$a_i - i$ – тая цифра договора ($0 \leq i < nd$)

Пример. Номер договора 07815. cv =17; Номер варианта $nv=(0+7+8+1+5) \bmod(17)+1=5$.

Каждый вариант кодируется шестизначным числом.

Первая цифра - язык программирования для которого проектируется транслятор:

1-PASCAL; 2- C; 3-PYTHON; 4-BASIC.

Вторая цифра - подмножество языка:

1-арифметические выражения целого типа, операторы присваивания, цикла с предусловием, бесформатного ввода-вывода;

2-арифметические выражения целого типа, операторы присваивания, условный, бесформатного ввода-вывода;

3-арифметические выражения вещественного типа, операторы присваивания, цикла с предусловием, бесформатного ввода-вывода;

4-арифметические выражения целого типа с индексированными и простыми переменными, операторы присваивания, цикла с постусловием, бесформатного ввода-вывода;

5-арифметические выражения целого типа, функции, операторы присваивания, условный, бесформатного ввода-вывода;

6-арифметические выражения целого типа, логические выражения операторы присваивания, цикла с предусловием, бесформатного ввода-вывода.

Третья цифра - тип промежуточного кода программы:

1-польская запись;

2-синтаксическое дерево;

3-тетрады.

Четвертая цифра - тип транслятора:

1-компилятор;

2-интерпретатор.

Пятая цифра - метод разбора:

1 - нисходящий с возвратами;

2 - рекурсивный спуск;

3 – нисходящий на основе автомата с магазинной памятью

4 - простое предшествование;

5 - расширенное предшествование.

Шестая цифра - язык программирования, на котором производится реализация транслятора:

1 - C;

2 - C++;

3 - C#;

4 - Python.

2.3 Варианты курсовых работ

Номер варианта рассчитывается следующим образом.

Обозначим n – количество цифр договора, N – номер варианта. Номер договора $N_d = a_1 a_2 \dots a_n$, где $a_i - i$ – та цифра номера договора, Например $N_d = 352$, $n=3$, $a_1=3$, $a_2=5$, $a_3=2$.

Формула для расчета номера варианта

$$N = (\sum_{i=1}^n (n + 1 - i) \times a_i) \bmod(120) + 1.$$

Для $N_d=352$, $N=(3*3+2*5+1*2) \bmod(120) + 1 = 12$

№ Варианта	Номера цифр кода варианта					
	1	2	3	4	5	6
1	1	5	2	1	3	2
2	2	6	3	2	4	1
3	3	1	1	1	5	2
4	4	2	2	2	1	3
5	1	3	3	1	2	4

6	2	4	1	2	3	5
7	3	5	2	1	4	6
8	4	6	3	2	5	1
9	1	1	1	1	1	2
10	2	2	2	2	2	3
11	3	3	3	1	3	4
12	4	4	1	2	4	5
13	1	5	2	1	5	6
14	2	6	3	2	1	1
15	3	1	1	1	2	2
16	4	2	2	2	3	3
17	1	3	3	1	4	4
18	2	4	1	2	5	5
19	3	5	2	1	1	6
20	4	6	3	2	2	1
21	1	1	1	1	3	2
22	2	2	2	2	4	3
23	3	3	3	1	5	4
24	4	4	1	2	1	5
25	1	5	2	1	2	6
26	2	6	3	2	3	1
27	3	1	1	1	4	2
28	4	2	2	2	5	3
29	1	3	3	1	1	4
30	2	4	1	2	2	5
31	3	5	2	1	3	6
32	4	6	3	2	4	1
33	1	1	1	1	5	2
34	2	2	2	2	1	3
35	3	3	3	1	2	4
36	4	4	1	2	3	5
37	1	5	2	1	4	6
38	2	6	3	2	5	1
39	3	1	1	1	1	2
40	4	2	2	2	2	3
41	1	3	3	1	3	4
42	2	4	1	2	4	5
43	3	5	2	1	5	6
44	4	6	3	2	1	1
45	1	1	1	1	2	2
46	2	2	2	2	3	3
47	3	3	3	1	4	4
48	4	4	1	2	5	5
49	1	5	2	1	1	6
50	2	6	3	2	2	1

51	3	1	1	1	3	2
52	4	2	2	2	4	3
53	1	3	3	1	5	4
54	2	4	1	2	1	5
55	3	5	2	1	2	6
56	4	6	3	2	3	1
57	1	1	1	1	4	2
58	2	2	2	2	5	3
59	3	3	3	1	1	4
60	4	4	1	2	2	5
61	1	5	2	1	3	6
62	2	6	3	2	4	1
63	3	1	1	1	5	2
64	4	2	2	2	1	3
65	1	3	3	1	2	4
66	2	4	1	2	3	5
67	3	5	2	1	4	6
68	4	6	3	2	5	1
69	1	1	1	1	1	2
70	2	2	2	2	2	3
71	3	3	3	1	3	4
72	4	4	1	2	4	5
73	1	5	2	1	5	6
74	2	6	3	2	1	1
75	3	1	1	1	2	2
76	4	2	2	2	3	3
77	1	3	3	1	4	4
78	2	4	1	2	5	5
79	3	5	2	1	1	6
80	4	6	3	2	2	1
81	1	1	1	1	3	2
82	2	2	2	2	4	3
83	3	3	3	1	5	4
84	4	4	1	2	1	5
85	1	5	2	1	2	6
86	2	6	3	2	3	1
87	3	1	1	1	4	2
88	4	2	2	2	5	3
89	1	3	3	1	1	4
90	2	4	1	2	2	5
91	3	5	2	1	3	6
92	4	6	3	2	4	1
93	1	1	1	1	5	2
94	2	2	2	2	1	3
95	3	3	3	1	2	4

96	4	4	1	2	3	5
97	1	5	2	1	4	6
98	2	6	3	2	5	1
99	3	1	1	1	1	2
100	4	2	2	2	2	3
101	1	3	3	1	3	4
102	2	4	1	2	4	5
103	3	5	2	1	5	6
104	4	6	3	2	1	1
105	1	1	1	1	2	2
106	2	2	2	2	3	3
107	3	3	3	1	4	4
108	4	4	1	2	5	5
109	1	5	2	1	1	6
110	2	6	3	2	2	1
111	3	1	1	1	3	2
112	4	2	2	2	4	3
113	1	3	3	1	5	4
114	2	4	1	2	1	5
115	3	5	2	1	2	6
116	4	6	3	2	3	1
117	1	1	1	1	4	2
118	2	2	2	2	5	3
119	3	3	3	1	1	4
120	4	4	1	2	2	5

3 ТЕОРЕТИЧЕСКИЕ ПОЛОЖЕНИЯ

В задачу курсовой работы входит проектирование транслятора программ, написанных на алгоритмическом языке высокого уровня.

Под трансляцией понимается перевод исходного текста программы в объектный код. Схема процесса трансляции приведена на рис. 1.



Рис.1. Схема работы транслятора

Процесс перевода включает следующие этапы:

- лексический анализ;
- синтаксический анализ;
- получение промежуточного кода программы;
- оптимизация кода;
- генерация объектного кода;

Одним из основных требований, предъявляемых к трансляторам, является их способность обрабатывать любые входные цепочки символов, в том числе и ошибочные. В последнем случае транслятор должен выводить диагностическую информацию о предполагаемых ошибках.

В целом, транслятор должен проводить анализ исходного текста программы, а затем синтез объектного кода.

3.1 Синтаксический анализатор.

Нисходящие методы разбора.

Алгоритм нисходящего разбора с возвратами.

По алгоритму нисходящего разбора строится синтаксическое дерево, начиная с корня, постепенно спускаясь до уровня предложения, как это показано на рис.2.

На рисунке приведены конечные стадии разбора и не показаны деревья, которые

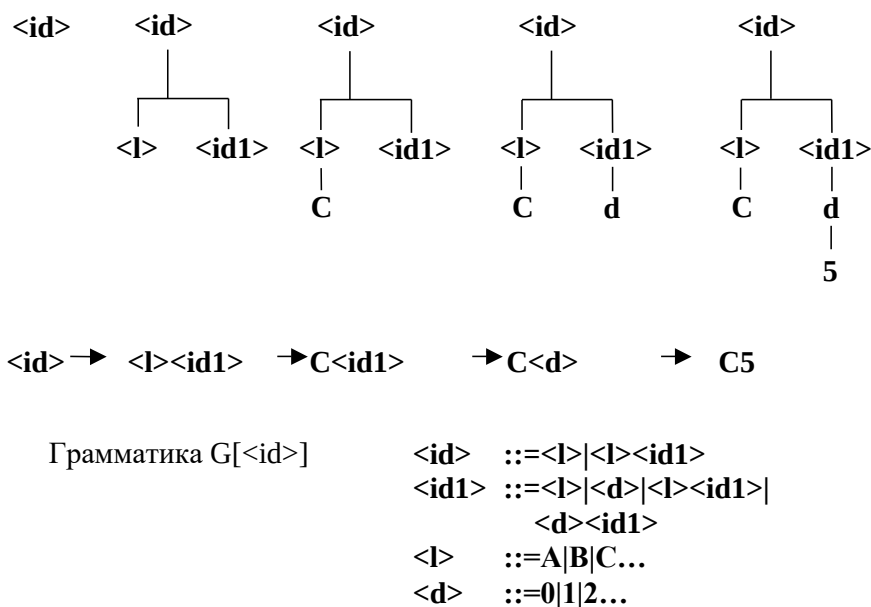


Рис. 2. Построение синтаксического дерева и вывода для цепочки **C5**

соответствуют локальным неудачным попыткам. Алгоритм разбора должен обладать той особенностью, что при неудачной попытке разбора он должен перебирать варианты разбора сентенциальной формы вплоть до первой удачной, либо до конца всего множества вариантов. Эффективность алгоритма повышается, если он в полной мере использует уже разобранные фрагменты входной цепочки.

Сущность сокращенного алгоритма разбора можно описать следующим образом. Пусть требуется произвести разбор цепочки x .

Предложение должно удовлетворять грамматике $G[Z]$, имеющей вид:

$Z ::= U | V | W$

$U ::= U_1 U_2 U_3 | U_4 U_5 U_6 \dots$

$V ::= V_1 V_2 V_3 \dots$

$W ::= W_1 W_2 W_3 \dots$

...

Задачей разбора является поиск вывода $Z \rightarrow +x$, где Z - начальный символ. Следовательно, первым непосредственным выводом должен быть вывод $Z \rightarrow U$ в соответствии с правилом $Z ::= U$. Если построить общий вывод, начинающийся с указанного непосредственного вывода нельзя, следует применить первым непосредственный вывод $Z \rightarrow V$ и т.д.. При выборе одного из возможных выводов, например $Z \rightarrow U$, ставится задача непосредственного вывода $U \rightarrow U_1 U_2 U_3$, а затем $U_1 \rightarrow +x_1$, где x_1 - некоторая подцепочка цепочки x , такая, что $x = x_1 x_2 \dots$. Если последний вывод произведен удачно, то выполняется вывод $U_2 \rightarrow +x_2$ и т.д.. Успех вывода $U_1 \rightarrow +x_1$ зависит от одновременного успеха множества конечных непосредственных выводов, порождаемых соответствующими правилами вывода и имеющих вид $P_i \rightarrow T_j$, где P_i - нетерминальный символ грамматики, а T_j - текущий терминальный символ входной строки. Успешность непосредственного вывода $P_i \rightarrow T_j$

зависит от того совпадает ли символ T_j с соответствующим текущим символом входной цепочки. Если непосредственный вывод удачен, то текущим символом становится следующий символ входной цепочки. Рассмотрим для примера случай, когда в приведенной выше грамматике правило для U имеет вид

$U ::= id+id|id-id,$

а входная цепочка " $a-b$ " после обработки сканером имеет будет представлена как $id-id$. Тогда вывод $U_1 \rightarrow +X_1$ условно выглядит как $id \rightarrow id$ и этот непосредственный вывод успешен, т.к. символ U_1 является терминальным и совпадает с текущим символом входной цепочки. Сообщение об успехе передается символу U , который рассматривает следующий текущий символ в правой части правила и порождает процесс вывода $U_2 \rightarrow +X_2$. Одновременно следующим текущим символом входной цепочки становится "-". Однако в этом случае сравнение порожденного терминального символа "+" и текущего символа входной цепочки "-" неуспешен. Следовательно символу U передается сообщение о неуспехе. Символ U возвращается на шаг назад и порождает новый вывод $U_1 \rightarrow +X_1$. Если породить новый вывод невозможно, то осуществляется переход к следующей альтернативе, т.е. порождается непосредственный вывод $U \rightarrow U_4 U_5 U_6$. В данном примере это будет вывод $U \rightarrow id-id$, который полностью успешен, следовательно символу U необходимо передать сообщение об успехе. Символ U продвигается на один символ вперед по своей текущей альтернативе и, в общем случае, порождает новый непосредственный вывод. В данном примере достигается конец альтернативы, что означает успех всего разбора в целом. Из изложенного следует сделать такие выводы:

- при возврате продвижение по текущей альтернативе производится относительно места откуда был произведен соответствующий вывод, поэтому при порождении вывода это место необходимо запоминать;
- если при возврате получено сообщение об успехе, то продвижение по текущей альтернативе производится на один символ в прямом направлении;
- если при возврате получено сообщение о неуспехе, то продвижение по текущей альтернативе производится на один символ в обратном направлении;
- порождение выводов производится в соответствии с правилами заданной грамматики.

Положительной чертой этого метода можно считать то, что каждый символ должен помнить лишь о своей цели, о своем родителе, о своих непосредственных потомках, а также о своем месте в грамматике и во входной цепочке. Происходит абстрагирование от точных сведений о том, что выполняется в других местах, т.е. в каждом сегменте программы или в подпрограмме необходимо заботится лишь о собственной входной и выходной информации.

Особенности реализация программы нисходящего разбора.

Задание грамматики. Один из способов задания состоит в представлении грамматики списком в одномерном массиве GRAMMAR таким образом, что каждое множество правил $U ::= x|y|...|z$ представлено как $Ux|y|...|z|$. То есть каждый символ занимает один элемент массива, за каждой правой частью U следует символ конца альтернативы – вертикальная черта |, а за последней правой частью U следует пара символов $|\$$ (конец альтернативы и конец правила). Таким образом, грамматика

$Z ::= E\#$
 $E ::= T+E|T$
 $T ::= F*T|F$
 $F ::= (E)|id$

будет выглядеть как $GRAMMAR = (ZE\#|SET+E|T|STF*T|F|SF(E)|id|\$)$.

Для порождения и уничтожения выводов в программе используется стек типа LIFO (Last-In-First-Out). Как правило для реализации стека используются массив **S** и счетчик **v**. При **v=0** стек пуст. При **v=n**, где **n>0**, в стеке находятся элементы **S(1), S(2),..., S(n)**, где **S(n)**–верхний элемент стека.

Каждый элемент стека соответствует одному порожденному непосредственному выводу и состоит из пяти компонент

(GOAL, i, FAT, SON, BRO), которые означают следующее.

GOAL – цель, т.е. символ, который подлежит разбору. Таким образом, в незакрытой в данный момент части входной цепочки предстоит найти такую голову, которая приводится к **GOAL**, и закрыть эту часть цепочки символом **GOAL**. **GOAL** передается элементу стека родителем.

i – индекс в массиве GRAMMAR, указывающий на тот символ в правой части правила для **GOAL**, с которым производится работа в данный момент.

FAT – имя родителя (номер элемента стека, соответствующего родителю).

SON – имя самого последнего из непосредственных выводов для текущей альтернативы.

BRO – предыдущего непосредственного вывода в текущей альтернативе.

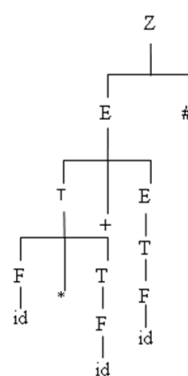
Нуль в любом из полей означает, что данный элемент отсутствует.

Массив GRAMMAR имеет 29 элементов:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
Z E # | \$ E T + E | T | \$ T F * T | F | \$ F (E) | i | \$

В качестве иллюстрации работы распознавателя на рис.3 приведено синтаксическое дерево и стек для предложения **id*id+id**.

Из содержимого стека видно, что имеется список непосредственных потомков каждого родителя и элементы этого списка "связаны" между собой. Стек в его окончательном виде не что иное, как внутренняя форма синтаксического дерева. Следует отметить, что в конце каждого предложения в соответствии с грамматикой используется разделитель. По нему можно судить о том, что окончен разбор всего предложения, а не только какой-то из его подцепочек.



СТЕК	GOAL	i	FAT	SON	BRO
1	Z	4	0	15	0
2	E	10	1	11	0
3	T	18	2	7	0
4	F	28	3	5	0
5	id	0	4	0	0
6	*	0	3	0	3
7	T	20	3	8	6
8	F	28	7	9	0
9	id	0	8	0	0
10	+	0	2	0	3
11	E	12	2	12	10
12	T	20	11	13	0
13	F	28	12	14	0
14	id	0	13	0	0
15	#	0	1	0	2

Рис. 3. Синтаксическое дерево и стек для выражения **id*id+id**

3.2 Требования к грамматикам для нисходящего разбора.

В алгоритме, описанном выше, есть недостаток, который проявляется, когда цель определена с использованием левосторонней рекурсии. Если **X**– цель, а первое же правило для **X** имеет вид **X::=X...**, то происходит бесконечное порождение непосредственных выводов **X→X...** Поэтому грамматика написана с применением правосторонней рекурсии вместо привычной левосторонней. Более предпочтительный способ избавится от прямой

левосторонней рекурсии – записывать правила, используя итеративные и факультативные цепочки. При этом правила преобразуются к следующему виду.

Вместо $E ::= E + T | T$ будет $E ::= T \{ + T \}$.

Вместо $T ::= T * F | T / F | F$ будет $T ::= F \{ * F / F \}$

На практике используются два принципа, на основании которых правила языка, включающие прямую левостороннюю рекурсию, преобразуются в эквивалентные правила, использующие итерацию.

Факторизация.

Если существуют правила вида

$U ::= x y | x w | \dots | x z$, то их надо заменить на

$U ::= x(y|w|\dots|z)$, где скобки (и) являются метасимволами.

Допустима факторизация и в более общей форме, такая как в арифметических выражениях. Например, если $y = fv$, $w = fm$ то $U ::= x(f(v|m))| \dots$. Следует отметить, что исходные правила $U ::= x|xy$ преобразуются к виду $U ::= x(y|\lambda)$, где λ – пустая цепочка. Когда бы не использовалось подобное преобразование, λ всегда помещается как альтернатива, так как мы принимается условие, что если цель – λ , то эта цель всегда сопоставляется. Помимо того что факторизация позволяет исключать прямую рекурсию, использование этого приема сокращает размеры грамматики и позволяет проводить разбор более эффективно.

Итерация.

После факторизации в грамматике останется не более одной правой части с прямой левосторонней рекурсией для каждого из нетерминалов. Если такая правая часть есть, необходимо выполнить следующее. Пусть $U ::= x|y|\dots|z|Uv$ – правила, у которых осталась леворекурсивная правая часть. Эти правила означают, что членом синтаксического класса U является x , y или z , за которыми ничего не следует, либо следует сколько-то v . Тогда правила преобразуются к виду $U ::= (x|y|\dots|z) \{v\}$.

После таких изменений должен измениться и алгоритм нисходящего разбора. Теперь алгоритм должен уметь обрабатывать альтернативы не только во всей правой части, но и в подцепочках, должен учитывать в своей работе существование пустой цепочки λ , должен уметь обрабатывать итерацию. Устранение прямой левосторонней рекурсии не позволяет устранить общую левостороннюю рекурсию. Таким образом, правила $U ::= Vx$ и $V ::= Uy|v$ дают вывод $U \rightarrow +Uyx$. Избавится от этого не так просто, но обнаружить такую ситуацию возможно, если использовать следующий подход. Из исходной грамматики исключаются все правила с левосторонней рекурсией. Символ U , получившейся в результате преобразования грамматики, может быть леворекурсивным тогда и только тогда, когда $U \text{ FIRST}^+ U$. Существуют алгоритмы проверки этого отношения.

3.3 Восходящие методы разбора

Простое предшествование.

Метод простого предшествования основан на том, что между двумя соседними символами $S_i S_{i+1}$ приводимой строки могут существовать только три отношения:

$S_i < \bullet S_{i+1}$ - отношение верно, если символ S_{i+1} - самый левый символ некоторой основы;

$S_i \bullet > S_{i+1}$ - отношение верно, когда S_i - символ, являющийся самым правым в некоторой основе;

$S_i \bullet = S_{i+1}$ - верно, если S_i и S_{i+1} принадлежат одной основе.

Эти отношения можно пояснить следующим образом. Для текущей цепочки

$\dots S_i < \bullet S_{i+1} \bullet = \dots \bullet = S_j \bullet > S_{j+1} \dots S_k \bullet > S_{k+1} \dots$

символы $S_{i+1} \dots S_j$ составляют основу.

Отношения $\langle \bullet, \bullet =, \bullet \rangle$ называются отношениями предшествования.

Отношения предшествования являются отношениями порядка, т.е. они зависят от порядка следования символов S_i и S_{i+1} . Если имеется отношение $S_i \bullet = S_{i+1}$ - это не означает, что $S_{i+1} \bullet = S_i$.

Если для каждой пары символов грамматики существует не более, чем одно отношение предшествования, то на каждом шаге синтаксического анализа можно выделить основу. Основой называется самая левая

группа символов, связанных отношениями предшествования вида:

$$\langle \bullet S_i \bullet = S_{i+1} \dots S_{j-1} \bullet = S_j \bullet \rangle.$$

Грамматика предшествования.

Пусть U, C, D - нетерминальные символы; X, Y, ω - любые цепочки.

Грамматикой предшествования называется такая грамматика класса 2, которая отвечает следующим двум требованиям:

- для каждой упорядоченной пары терминальных и нетерминальных символов должно выполняться не более, чем одно из трех отношений предшествования, определяемых следующим образом.

$S_i \bullet = S_j$, если существует правило $U ::= X S_i S_{i+1} Y$;

$S_i \bullet < S_j$, если существует правило $U ::= X S_i D Y$ и существует вывод из D цепочки $S_i \omega$,

$S_i \bullet > S_j$, если существует правило $U ::= X C S_{i+1} Y$ и существует вывод $C \rightarrow^* \omega S_i$.

- разные порождающие правила должны иметь разные правые части.

Например, правило для списка идентификаторов -

$id_list ::= id | id_list, id$

и правило для элемента выражения

$F ::= id | (E) \dots$

имеют одинаковые правые, но разные левые части. Устранить отрицательные последствия таких правил можно лишь преобразовав надлежащим образом грамматику. Например, если ввести несколько иной смысл в общепринятое понятие списка элементов, то правило грамматики примет вид.

$id_list ::= id, id | id_list, id$

Распознаватель метода простого предшествования.

Алгоритм разбора, основанный на отношениях предшествования, заключается в следующем. Символы входной строки поочередно переписываются в стек до тех пор, пока между символом на вершине стека S_i и очередным символом входной строки S_i не появится отношение $\bullet >$. Это признак того, что в стеке находится основа. Стек в этом случае просматривается в направлении от вершины к началу до тех пор, пока между двумя очередными символами стека не появится отношение $\bullet <$, т.е. $S_{k-1} \bullet < S_k$. В этом случае часть стека от вершины S_i до символа S_k является основой. После этого среди порождающих правил отыскивается правило

$U ::= S_k \dots S_i$,

и последовательность символов в стеке $S_k \dots S_i$ заменяется на найденный символ U . Далее процесс повторяется аналогично.

Если на некотором шаге процесса разбора обнаружится, что между символами S_p и S_q не существует отношения предшествования, то это означает, что во входной цепочке допущена синтаксическая ошибка.

Матрица предшествования.

Матрица предшествования формируется на предварительном этапе проектирования транслятора и должна содержать отношения предшествования между всеми символами грамматики. Обозначим $L(U)$ - множество самых левых символов относительно нетерминала U . Формально это множество определяется следующим образом.

$$L(U) = \{S \mid \exists (U \rightarrow *Sz)\},$$

где S -любая строка, возможно пустая, $\exists$ -квантор существования.

Это множество можно определить рекурсивно:

$$L(U) = \{S \mid \exists (U ::= S...) \cup [\exists (U ::= U'...) \cap S \in L(U')]\}.$$

Аналогично определяется множество $R(U)$.

С использованием множеств $R(U)$ и $L(U)$ отношения предшествования принимают вид пригодный для практического использования.

$$S_i = \bullet S_j \Leftrightarrow \exists (U ::= \dots S_i S_j \dots)$$

$$S_i < \bullet S_j \Leftrightarrow \exists (U ::= \dots S_i D \dots) \cap S_j \in L(D)$$

$$S_i \bullet > S_j \Leftrightarrow \exists (U ::= \dots D S_j \dots) \cap S_i \in R(D)$$

где D -нетерминальный символ.

Построение $L(U)$ и $R(U)$ происходит следующим образом.

Для каждого нетерминального символа грамматики отыскивается

правило, в левой части которого находится символ U . В множество $L(U)$ включаются все самые левые символы правых частей правил для U . В множество $R(U)$ включаются все правые символы из правил для U .

Рассматривается полученное множество $L(U)$. Если оно содержит нетерминальные символы $U', U'', \dots$, то левые символы для $U', U'', \dots$ заносятся в $L(U)$.

Аналогично, если $R(U)$ содержит нетерминальный символ $U_1, U_2, \dots$, то в $R(U)$ включаются все правые символы для $U_1, U_2, \dots$. Процесс продолжается до тех пор, пока множества $L(U)$ и $R(U)$ не перестают изменяться.

Пример. Для приведенной ниже грамматики показаны шаги формирования множеств левых и правых символов:

$$Z ::= \#E\#$$

$$E ::= E + E \mid T$$

$$T ::= T * F \mid F$$

$$F ::= (E) \mid id$$

Шаг 1		
U	L(U)	R(U)
Z	#	#
E	E,T	T
T	T,F	F
F	(,id	),id

Шаг 2		
U	L(U)	R(U)
Z	#	#
E	E,T,F	T,F
T	T,F,(,id	F
F	(,id	),id

Шаг 3		
U	L(U)	R(U)
Z	#	#
E	E,T,F,(,id	T,F,),id
T	T,F,),id	F,),id
F	(,id	),id

Построение матрицы предшествования.

Матрица предшествования имеет размерность $n \times n$, где n - количество символов грамматики. Символы грамматики записываются в первый столбец и первую строку, которые называются входными для отыскания отношений предшествования. Первый символ отношения выбирается из входного столбца, второй символ отношения - из входной строки.

На первом этапе отыскиваются символы, между которыми существует отношение $=\bullet$. Это отношение полностью определено правилами грамматики. Для найденных символов в соответствующую ячейку матрицы предшествования записывается символ отношения.

На втором этапе отыскиваются символы, между которыми существует отношение $<\bullet$. Эти символы определяются путем просмотра правых частей правил грамматики и затем использования определяющего соотношения. Символ отношения записывается в соответствующую ячейку матрицы.

На третьем этапе аналогичным образом отыскивается отношение $\bullet>$.

Пример матрицы предшествования для приведенной выше грамматики показан в табл.1.

Таблица 1. Матрица предшествования

$S \mid S_i$	E	T	F	+	*	(	)	id	#
E				$=\bullet$			$=\bullet$		$=\bullet$
T				$\bullet>$	$=\bullet$		$\bullet>$		$\bullet>$
F				$\bullet>$	$\bullet>$		$\bullet>$		$\bullet>$
+		$\frac{=\bullet}{<\bullet}$	$<\bullet$			$<\bullet$		$<\bullet$	
*			$=\bullet$			$<\bullet$		$<\bullet$	
(		$\frac{=\bullet}{<\bullet}$	$<\bullet$	$<\bullet$			$<\bullet$		$<\bullet$
)				$\bullet>$	$\bullet>$		$\bullet>$		$\bullet>$
id				$\bullet>$	$\bullet>$		$\bullet>$		$\bullet>$
#		$\frac{=\bullet}{<\bullet}$	$<\bullet$	$<\bullet$			$<\bullet$		$<\bullet$

Из матрицы предшествования видно, что данная грамматика не является грамматикой предшествования, так как существует неоднозначность отношений предшествования. Неоднозначность отношений предшествования для данной грамматики является следствием того, что в методе простого предшествования используется минимальный контекст.

Для устранения неоднозначности отношений предшествования следует использовать более глубокий контекст.

Так, в контексте $...(E)...$ справедливо отношение $(=\bullet E$ и возможна прямая редукция: $(E) \rightarrow E$.

В контексте $...(E+T)...$ или $...+T^*F...$ между $($ и E имеется отношение $<\bullet$ и необходима прямая редукция $E+T \Rightarrow E$, а между $+$ и T справедливо отношение $<\bullet$ и, следовательно, необходима редукция $T^*F \Rightarrow T$.

В данном случае формально неоднозначность отношений предшествования вытекает из того, что, например, $($ и E встречаются рядом в правой части, а с другой стороны E принадлежит $L(E)$, то есть $L(E)$ рекурсивно для символа E . Избежать неоднозначности отношений предшествования можно, используя метод стратификации (отделения), который устраняет рекурсивность множества $L(U)$ или $R(U)$ для тех символов U , для которых отношения неоднозначны.

Например, после введения новых правил

$$E::=E'$$

$$E'::=E+T|T$$

множество левых символов $L(E)=\{E', T, F, (, id\}$ уже не содержит символа E .

Аналогично вводя стратификацию для символа T можно записать:

$$Z::=\#E$$

$$T::=T'$$

$$E::=E'$$

$$T'::=T^*F|F$$

$$E'::=E'+T|T$$

$$F::=(E)|id.$$

Расширенное предшествование.

Практическое применение метода простого предшествования ограничено вследствие появления неоднозначности отношений предшествования, которые часто встречаются при проектировании грамматик реальных языков программирования. Общего алгоритма приведения исходной грамматики к грамматике, отвечающей требованиям метода простого предшествования не существует. Поэтому разработаны методы, позволяющие преодолеть отрицательные эффекты неоднозначных отношений. Одним из таких методов является метод расширенного предшествования. Ниже рассмотрены основные положения, связанные с использованием расширенного предшествования (1,2)(2,1).

Основная идея метода расширенного предшествования состоит в следующем. Пусть между символами S_j и S_{j+1} существует неоднозначное отношение, например, $S_j <\bullet S_{j+1}$ и $S_j =\bullet S_{j+1}$. Требуется определить, какое отношение нужно взять в контексте $...S_j S_{j+1} S_{j+2}...$. Если справедливо отношение $<\bullet$ и грамматика однозначна, то должно существовать правило, которое начинается символами $S_{j+1}S_{j+2}...$, то есть правило имеет вид: $U::=S_{j+1}S_{j+2}...$. Если такого правила не существует, то справедливым будет отношение $=\bullet$. Аналогично, если между символами S_j и S_{j+2} существуют отношения $S_{j+1} =\bullet S_{j+2}$ и $S_{j+1} \bullet > S_{j+2}$ и требуется определить, какое из них необходимо взять в контексте $...S_j S_{j+1} S_{j+2}...$ то есть является ли S_{j+1} правым концом основы, то отношение $\bullet >$ будет верным, если существует правило вида $U::=...S_j S_{j+1}$. В противном случае надо брать другое конфликтное отношение.

С целью сокращения просмотров грамматики при определении верного отношения предшествования предварительно заготавливается вспомогательная информация в виде множеств левых и правых троек. Множество левых троек используется для выделения левого конца основы, множество правых троек - для выделения правого конца основы.

Множество левых троек LT содержит такие тройки символов $S_1 S_2 S_3$, что символы $S_2 S_3$ начинают правую часть какого-либо правила, а между символами S_1 и S_2 имеется неоднозначность отношений $(=\bullet, <\bullet)$.

Множество правых троек RT содержит такие тройки символов $\underline{S_1S_2S_3}$, что символы $\underline{S_2S_3}$ заканчивают правую часть какого-либо правила, а между символами $\underline{S_1}$ и $\underline{S_2}$ имеется неоднозначность отношений ($=\bullet, \bullet>$).

Составление таблицы правых троек.

Просмотреть матрицу предшествования и найти пары символов $\underline{S_2S_3}$, для которых существуют конфликтные отношения ($=\bullet, \bullet>$).

Для каждой выявленной пары символов $\underline{S_2S_3}$ просмотреть столбец $\underline{S_2}$ матрицы предшествования и найти все символы $\underline{S_1}$, связанные с $\underline{S_2}$ отношением $\underline{S_1}=\bullet\underline{S_2}$.

Просмотреть порождающие правила. Если существует правило, которое заканчивается парой $\underline{S_1S_2}$, т.е. $U::=\dots\underline{S_1S_2}$, то тройка $\underline{S_1S_2S_3}$ записывается в таблицу правых троек.

Составление таблицы левых троек.

Просмотреть матрицу предшествования и найти пары символов $\underline{S_1S_3}$, для которых существуют конфликтные отношения ($=\bullet, <\bullet$).

Для каждой выявленной пары символов $\underline{S_1S_2}$ просмотреть строку $\underline{S_2}$ матрицы предшествования и найти все символы $\underline{S_3}$, связанные с $\underline{S_2}$ отношением $\underline{S_2}=\bullet\underline{S_3}$.

Просмотреть порождающие правила. Если существует правило, которое начинается парой $\underline{S_2S_3}$, т.е. $U::=\underline{S_2S_3}\dots$, то тройка $\underline{S_1S_2S_3}$ записывается в таблицу правых троек.

Для примера, приведенного выше видно, что неоднозначных отношений ($=\bullet, \bullet>$) не существует, поэтому множество правых троек будет пустым.

Множество левых троек $LT=\{ \# E + , (E + , + T * \}$

3.4 Семантические программы

В результате синтаксического и лексического анализов производится некоторое в общем случае неэквивалентное преобразование исходного текста в промежуточный код. Семантические же программы осуществляют перевод программы в форму, эквивалентную исходному тексту. Семантическая обработка, как правило, связывается с синтаксической и предназначена для формирования одного из промежуточных кодов программы. На практике используются следующие промежуточные коды: польская запись; тетрады и триады. Этап семантической обработки необходим для упрощения задачи перевода программы в объектный код.

Формирование польской записи для исходной программы.

Рассмотрим восходящий метод разбора. Синтаксический распознаватель всякий раз, когда найдена основа, и её можно привести к нетерминальному символу U , вызывает программу, связанную с правилом $U::=x$, где x - найденная основа. Эта программа осуществляет семантическую обработку всех символов цепочки x и формирует часть символов польской записи, которая имеет непосредственное отношение к x . При этом, с учетом того, что основа редуцируется при каждом её нахождении, если в основе встречается некоторый нетерминал V , то часть польской записи, включающая подцепочку, которая приводится к V , уже сгенерирована.

Предположим, что генерируемая польская запись находится в одномерном массиве P . Целая переменная J (начальное значение равно единице) содержит индекс, который указывает на первый свободный элемент массива P . Каждый элемент массива может содержать один символ: идентификатор или оператор. Пусть вызванная семантическая программа имеет доступ к символам основы $S[1], S[2], \dots, S[i]$. Рассмотрим программу, связанную с правилом $E_1::=E_2+T$. Если данная семантическая программа вызвана, то польская запись для E_2 и T уже получена, т. е. она содержит $\langle \text{код для } E_2 \rangle \langle \text{код для } T \rangle$.

Кроме того символ, который находится правее **T** в исходной программе, является терминальным. Следовательно от данной семантической программы требуется только занести символ '+' в польскую запись ...<код для **E₂**><код для **T**>+. Т.е. инфиксная запись **E₂+T** переводится в польскую запись **E₂T+**.

Итак, семантическая программа имеет следующий вид:

P[J]:='+'; J:=J+1.

Семантическая программа, связанная с правилом **F::=id**, имеет вид

P[J]:=S[i]; J:=J+1,

где **S[i]** - это вершина синтаксического стека.

Семантическая программа, связанная с правилом **F::=(E)**, никаких действий не производит, т. к. цепочка для **E** уже сформирована, а приоритет операции в польской записи задается порядком следования операндов и операторов, т. е. без применения скобок.

Все типы семантических программ для грамматики алгебраических выражений приведенной выше можно свести в следующую таблицу 2.

Таблица 2. Семантические программы

N	Правило	Семантическая программа
1	Z::=E	нет
2	E::=T	нет
3	E::=E+T	P[J]:='+'; J:=J+1;
4	E::=E-T	P[J]:='-'; J:=J+1;
5	E::=-T	P[J]:='@'; J:=J+1;
6	T::=F	нет
7	T::=T*F	P[J]:='*'; J:=J+1;
8	T::=T/F	P[J]:='/'; J:=J+1;
9	F::=id	P[J]:=S[i]; J:=J+1;
10	F::=(E)	нет

Формирование промежуточного кода в виде тетрад и триад имеет некоторые особенности, связанные с сохранением семантической информации при редукции основы, остальные принципы аналогичны тем, по которым формируется польская запись.

Генерация объектного кода.

Для генерации объектного кода необходимо заготовить тексты программ на объектном языке, выполняющих действия полностью соответствующие по смыслу вызываемым семантическим программам. В указанных заготовках необходимо предусмотреть поля, в которые при генерации заносится конкретизирующая информация, такая, например, как имена переменных.

В качестве объектного кода используется язык Макроассемблера.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Системное программное обеспечение /А.В. Гордеев, А.Ю. Молчанов. – СПб.: Питер, 2001. – 736 с.
2. Бек Л. Введение в системное программирование. - М.:Мир,1988. - 448 с.
3. Лебедев В.М. Введение в системы программирования. М.: Статистика, 1975. - 312 с.
4. Ахо А., Ульман Дж. Теория синтаксического анализа, перевода и компиляции. - М.: Мир, 1978, в 2-х т.
5. Грис Д. Конструирование компиляторов для цифровых вычислительных машин. - М.: Мир, 1975. - 544 с.

6. Хопгуд Ф. Методы компиляции. - М.: Мир, 1972.
7. Вайнгаартен Ф. Трансляция языков программирования. - М.: Мир, 1977. - 190 с.
8. Кнут Д. Искусство программирования для ЭВМ. т.1 Основные алгоритмы. Перевод с английского. - Москва: Мир, 1976. - 735 с.
9. Мартин Дж. Организация баз данных в вычислительных системах: Перевод с английского. - Москва: Мир, 1980. - 664 с.
10. Зиглер К. Методы проектирования программных систем: Перевод с английского. - Москва: Мир, 1985. - 32 с. ил.